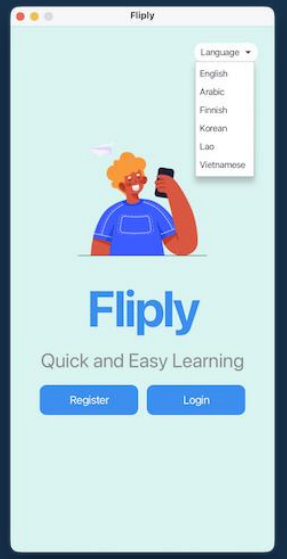
Software Engineering Project Presentation

Fliply

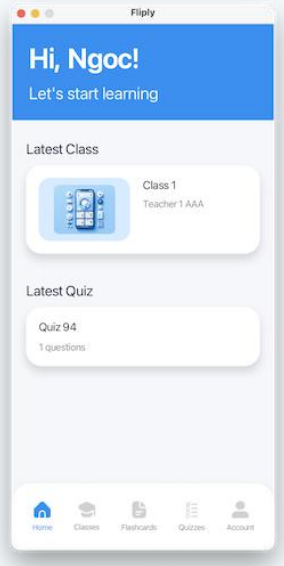
Online Flashcard System for Study



Project members: Ngoc Nguyen • Thanh Nguyen • Nhut Vo • Hoang Vu

Spring 2026

Content



Relevant Links

Repository, planning board, and documentation

GitHub

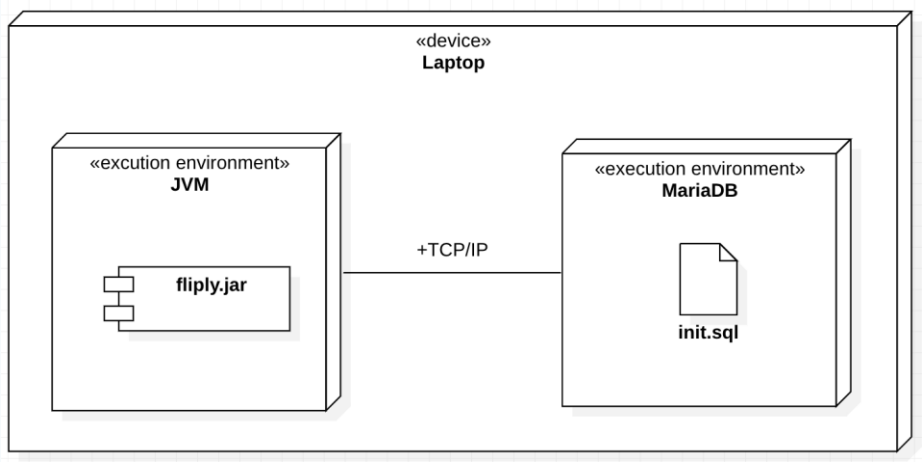
github.com/nguyngc/fliply

Trello

trello.com/w/sep1_group3/home

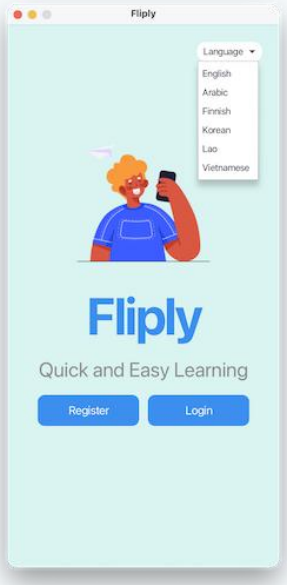
Documentation hub

- README links to sprint reports, diagrams, screenshots, QA evidence.
- Documents folder contains the main course artifacts.
- Status and course feedback can be discussed near the end.



Introduction

Problem and motivation



Problem

- Study material, flashcards, quizzes and progress are often split between many tools.
- Teachers need a simple way to publish content and follow learning progress.
- Multilingual students benefit from localized UI and localized study content.

Fliply solution

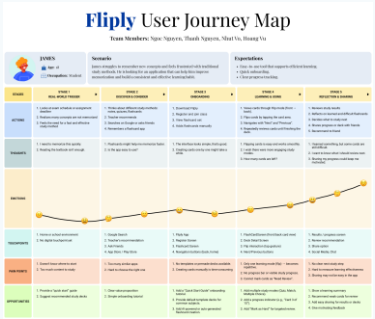
A JavaFX desktop application for authentication, classroom management, flashcards, quizzes, progress tracking and multilingual learning.

Project Vision and Goals

What Fliply aims to achieve

Vision statement

Build a structured learning platform where teachers publish study material and students practise it through flashcards, quizzes and classroom-based content.



Short-term

- Core student and teacher workflows
- Reliable build and database setup
- Basic localization

Long-term

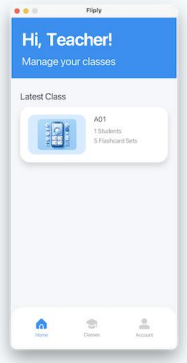
- More learning analytics
- Better classroom collaboration
- Easier deployment

Success criteria

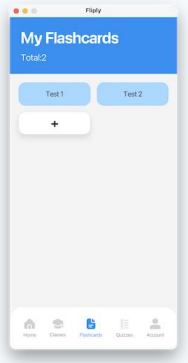
- Automated tests pass
- QA issues documented and fixed
- Final documentation complete

Application Features and Usage

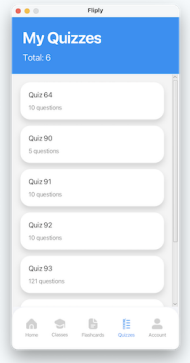
Core screens for a short demo



Teacher home



Flashcard set



Quiz mode

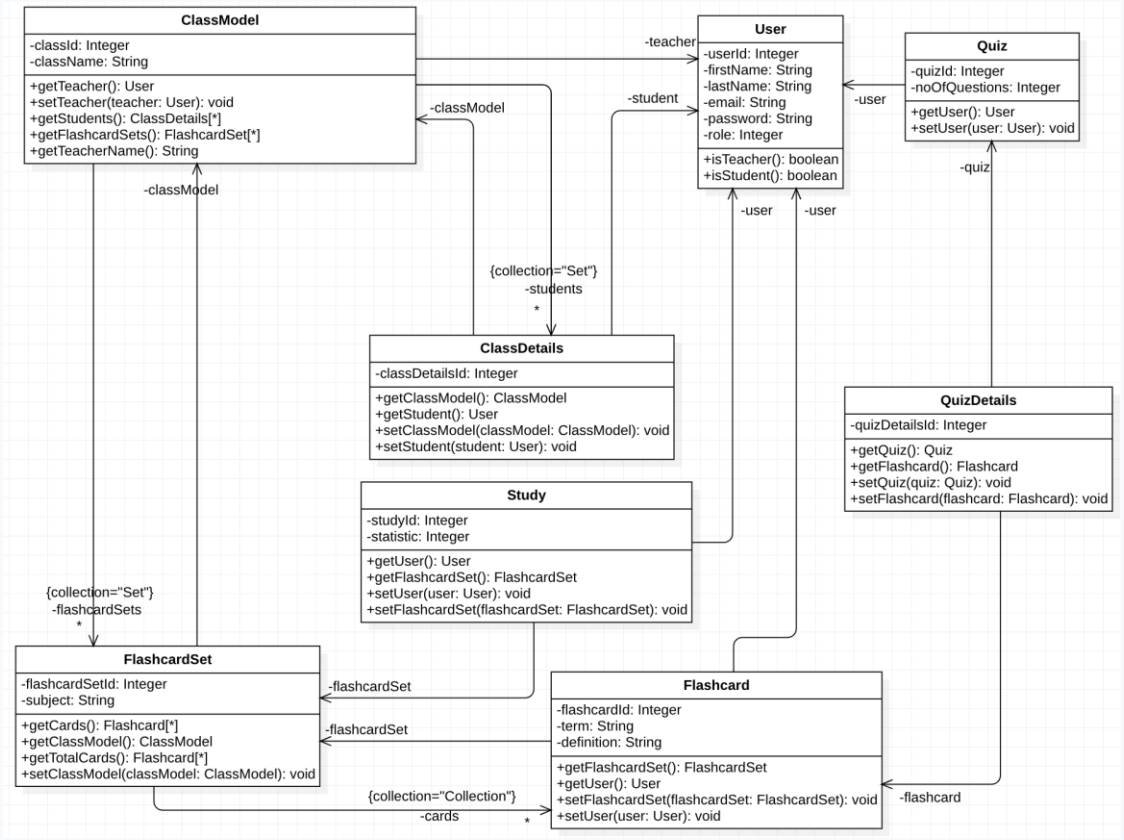
- Register/login as student or teacher
- Teacher creates classes and flashcard sets
- Student studies flashcards and answers quizzes
- System tracks progress and statistics

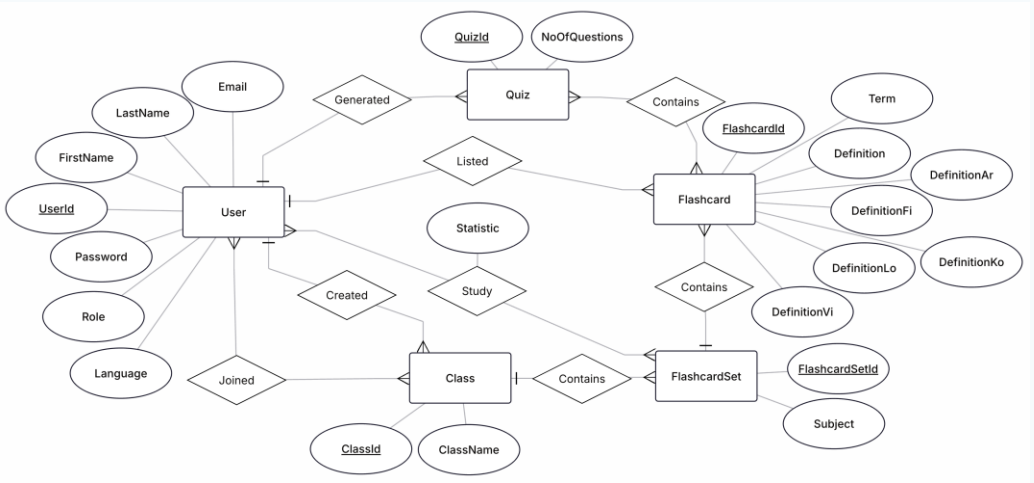
Demo flow: Login → select role screen → open class → review flashcards → take quiz → check result / change account language.

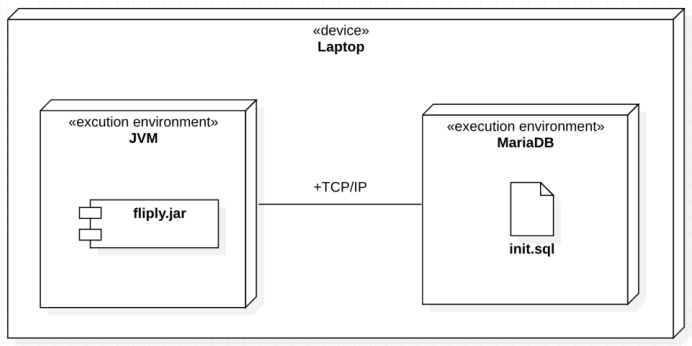
Demo

Software Architecture and Technologies Used

MVC-like JavaFX app with JPA/Hibernate persistence



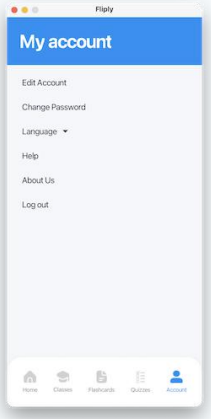




Java 21 • JavaFX/FXML • Maven • MariaDB • JPA/Hibernate • JUnit 5 • Mockito • JaCoCo • Docker • Jenkins • SonarQube

Localization

UI ResourceBundle + localized database content



UI localization

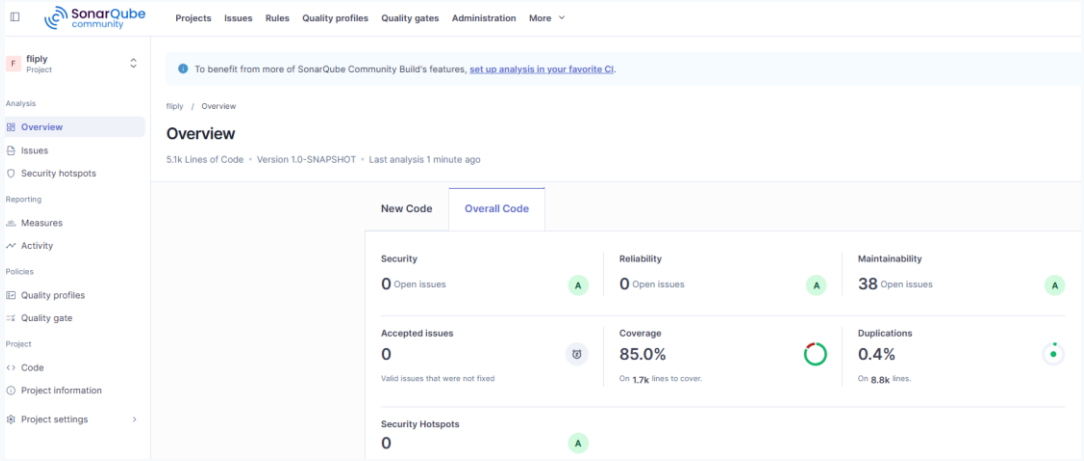
- Resource bundles: EN, VI, LO, KO, FI, AR
- Language preference stored per user
- Screen reloads after locale change

Database localization

- Flashcards store definitions in multiple languages
- Fallback to English if target language is missing
- CSV import supports multilingual definitions

Quality Assurance

Automated checks + manual validation




```
[INFO] Results:
[INFO]
[WARNING] Tests run: 273, Failures: 0, Errors: 0, Skipped: 6
[INFO]
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ fliply ---
[INFO] Building jar: C:\Users\hiryu\IdeaProjects\fliply\target\fliply-1.0-SNAPSHOT.jar
[INFO]
[INFO] --- jacoco:0.8.14:report (report) @ fliply ---
[INFO] Loading execution data file C:\Users\hiryu\IdeaProjects\fliply\target\jacoco.exec
[INFO] Analyzed bundle 'fliply' with 72 classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  31.706 s
[INFO] Finished at: 2026-04-14T13:05:00+03:00
[INFO] ------------------------------------------------------------------------
```

60+ manual test cases

~70% reported coverage

12 bugs fixed

Tools: JUnit 5, Mockito, JaCoCo, Jenkins, Docker, SonarQube, manual test cases and heuristic evaluation.

Functional and Non-functional Testing

End-to-end checks and heuristic analysis

Functional testing

- Register / login / role-based access
- Create class and manage flashcards
- Student study flow and quiz flow
- Input validation and confirmation dialogs
- Localized error messages

Result: main workflows passed after bug fixes in validation, deletion and quiz question count handling.

Non-functional testing

- Nielsen heuristic evaluation
- Readability and consistency checks
- Language switching and RTL check
- Static analysis with SonarQube
- Build reproducibility with Docker/Jenkins

Result: mostly positive usability; issues were documented with severity and improvement suggestions.

Lessons Learned

What the team improved through the project

Architecture	Separate UI controllers, services and persistence responsibilities.
Localization	Plan UI and database localization early because it affects views and data model.
DevOps	Docker and Jenkins help reproducibility, but JavaFX GUI apps need extra setup.
Testing	Manual test cases found workflow issues that unit tests alone did not catch.

Main challenge: integrating JavaFX, database, localization, testing and deployment into one stable final delivery.

Questions?

Thank you for listening

GitHub: github.com/nguyngc/fliply

Team: Ngoc Nguyen • Thanh Nguyen • Nhut Vo • Hoang Vu